

Project Proposal – Fleet Fuel Optimization Tool (Option 25)

Data Structures and business applications

Rayyan Asim(23i-5002), Sohaib Safdar(23i-5030)

BSBA-5B

1. Project Title & Selected Option:

The selected project for this semester is Option 25: Fleet Fuel Optimization Tool, which focuses on analyzing and optimizing fuel consumption data for business fleets using arrays, sorting, heaps, and searching techniques.

2. Problem Statement:

The Fleet Fuel Optimization Tool addresses a key problem faced by logistics, transportation, and delivery companies: the difficulty of efficiently monitoring and comparing vehicle fuel consumption. Many fleet-based businesses rely on manual logs or simple spreadsheets, which makes it slow and inconvenient to find specific vehicle data, identify fuel-inefficient vehicles, or detect irregular performance patterns. This leads to higher operational costs, inefficient route assignments, and delayed maintenance decisions. The project simulates a real-world scenario commonly encountered in logistics firms, courier services, inter-city transport companies, and organizations that manage large operational fleets. By creating a structured system that organizes fuel data, the project aims to simplify analysis and improve the accuracy and speed of fleet monitoring.

3. Business Impact:

This system offers strong practical value by improving decision-making and operational efficiency. Through efficient searching, managers can instantly retrieve any vehicle's information without scanning through many records. Sorting the fleet by fuel efficiency or consumption provides clear comparisons that support better route planning, driver performance evaluation, and resource allocation. The use of heaps allows the system to quickly identify the most fuel-inefficient vehicles, enabling businesses to prioritize them for inspection or maintenance and reduce unnecessary fuel usage. Overall, the tool makes fleet management smarter and more cost-effective by providing organized, quick, and data-driven insights that directly impact operational performance and financial savings.

4. Data Structures to use in the project:

The project will use several data structures that are essential for efficient data management. Arrays will store structured records of each vehicle, including fuel usage, distance, efficiency

scores, and service dates. Sorting algorithms will help generate ordered lists that highlight the best and worst-performing vehicles. A priority queue implemented with a heap will retrieve the highest or lowest efficiency vehicles instantly, making it easier to identify those requiring immediate attention. Searching techniques, either linear or binary, will be used to quickly locate specific vehicle IDs or retrieve performance logs. Each structure plays an important role: arrays allow quick indexing, sorting provides clear ranked insights, heaps handle priority decisions rapidly, and searching enables immediate lookup without scanning entire datasets. Together, these data structures ensure fast processing, accurate analysis, and effective business-level insights.

5. Implementation Plan:

The project will begin by taking vehicle input data and storing it in arrays. Each entry will include vehicle details such as ID, total distance, fuel used, and efficiency calculations. Users will be able to insert new fuel logs, update distances, or modify service information. Searching functions will allow retrieval of specific vehicle data, while sorting functions will create lists ordered by efficiency or fuel consumption. A heap structure will be used to build priority-based results, such as identifying the top five most fuel-inefficient vehicles. Algorithms such as Python's built-in sorting (Timsort), linear or binary search, and heap operations (heapq) will be utilized. Edge cases such as duplicate IDs, invalid inputs, zero fuel values, empty data structures, and incorrect entries will be handled through validation. The general program flow is: input data, then compute metrics, then we apply sorting/heap operations and lastly, we display insights to the user.

6. Expected Output & Features:

The system will produce several meaningful outputs, including sorted lists of vehicles based on efficiency or fuel consumption, priority-based results showing the most and least fuel-efficient vehicles, and fast search results for vehicle details. It will also generate summary insights such as fleet averages and highlight vehicles that may need maintenance due to decreasing fuel efficiency. These outputs not only simplify operational analysis but also help businesses reduce costs, assign routes intelligently, and identify potential performance issues early.

7. Tools & Technologies:

The project will be implemented in Python, using VS Code, PyCharm, or another preferred IDE. Python's built-in heapq library will be used for heap operations, and Python's default sorting algorithm (Timsort) will be used for efficient sorting tasks. Optional libraries like tabulate may be used to present clean output tables, and basic file handling may be included to store and load vehicle logs if necessary.